Projektuppgift

Introduktion till webbutveckling med HTML, CSS och JavaScript (DT224G)

Isac Larsson

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Isac Larsson, isla2401@student.miun.se
Utbildningsprogram: TWEUG, Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

1 Sammanfattning

En webbsida till den fiktiva nöjesparken Laxåland har tagits fram för att företaget ska kunna nå ut med information till blivande besökare. Fokus har legat på responsiv design, funktionalitet och tillgänglighet.

Den färdiga webbplatsen innehåller information om nöjesparken, öppettider, vägbeskrivning och vanliga frågor. Den har också en sida för att se biljetter och spara dessa i varukorgen. Utöver detta finns en mobilmeny som kan öppnas och stängas, ett kontaktformulär som valideras med JavaScript och en varukorg där sparade biljetter visas och kan tas bort.

Webbplatsen har testats med olika verktyg för att säkerställa god tillgänglighet och funktion på olika skärmstorlekar och i olika webbläsare.

Koden har versionshanterats med Git och webbplatsen är publicerad med hjälp av GitHub Pages.

Länk till den färdiga webbplatsen finns här:
https://isla2401.github.io/dt224g-projekt/

2 Innehållsförteckning

1 Sammanfattning 1
2 Innehållsförteckning 2
3 Introduktion 3
4 Teori 4
5 Metod 6
5.1 Förarbete 7
5.2 Innehåll och struktur 7
5.3 Layout och responsiv design 7
5.4 Funktionalitet 7
5.5 Utvecklingsmiljö och versionshantering 8
5.6 Testning 8
6 Konstruktion 8
6.1 Förarbete och design 9
6.2 Tillgänglighet 10
6.3 Struktur 10
6.4 Navigation 11
6.5 Utseende och layout 11
6.6 Funktionalitet 12
6.6.1 Validering av kontaktformulär 12
6.6.2 Visa och dölja innehåll 12
6.6.3 Spara biljetter till varukorg 13
6.6.4 Dynamiskt innehåll 13
6.7 Versionshantering och publicering 13
6.8 Testning 14
7 Resultat 14
8 Slutsatser 16
8.1 Måluppfyllelse 17
8.2 Förklaring till problem 17
8.3 Diskussion kring dynamiskt innehåll 17
8.4 Förbättringsmöjligheter 18
8.5 Vidareutveckling 18
Källförteckning 18

3 Introduktion

Den fiktiva nöjesparken Laxåland behöver en webbplats för att nå ut med information till kunder som vill besöka parken. Webbplatsen ska innehålla information om parken, priser, öppettider, vägbeskrivning och vanliga frågor. Det ska också vara enkelt att ta kontakt med företaget.

Webbplatsen ska vara interaktiv och ha funktionerna:

- En mobilmeny som går att öppna och stänga.
- Undermenyer i huvudmenyn för att lättare hitta information på en sida med mycket innehåll.
- Ett formulär där besökare kan skicka frågor till företaget. Formuläret ska valideras och ge tydlig återkoppling om något fält inte är korrekt ifyllt.
- Möjlighet att lägga biljetter i varukorgen.
- Visa varukorgens innehåll och ta bort produkter ur den.
- Visa och dölja svar på vanliga frågor. Detta för att ge en tydligare överblick över innehållet och gör att användaren kan välja vilket innehåll den vill se.

Företagets huvudsakliga målgrupp är personer som planerar att besöka nöjesparken. Till denna grupp hör främst barnfamiljer, men innefattar även andra personer som är intresserade av nöjesparker och åkattraktioner.

Webbplatsen ska vara estetiskt tilltalande och ha en design som signalerar lekfullhet. Webbplatsen ska fungera på olika skärmstorlekar och vara enkel att navigera på. Den ska också vara tillgänglig för användare med funktionsnedsättningar som exempelvis nedsatt syn och motorik.

4 Teori

HTML (HyperText Markup Language) är ett märkspråk som beskriver en webbsidas struktur och innehåll. I ett HTML-dokument används olika element, exempelvis rubriker, bilder och länkar, för att tala om för webbläsaren hur den ska visa innehållet. [1]

CSS (Cascading Style Sheets) är ett språk som används för att definiera hur innehållet på en webbplats ska se ut och presenteras. Det kan användas för att exempelvis ändra typsnitt, bakgrundsfärg och avstånd mellan element. [2]

JavaScript är ett programmeringsspråk som används för att skapa interaktivitet och funktionalitet på webbplatser. [3] JavaScript kan till exempel användas för att:

- Köra kod när en viss händelse inträffar. Detta görs med hjälp av **event handlers** (på svenska *händelsehanterare*). Ett **event** kan exempelvis vara att användaren trycker på en knapp eller att ett visst innehåll har laddats in.
- Modifiera **DOM** (Document Object Model), vilket är en trädstruktur över innehållet på en webbsida. Med JavaScript kan innehåll i DOM läggas till, tas bort eller ändras. [4]

Responsiv webbdesign innebär att webbplatsens layout och innehåll anpassar sig efter storleken på skärmen som den visas på [5]. För att skapa en responsiv design används flera arbetssätt och tekniker:

- **Mobile first** är ett arbetssätt som innebär att man börjar designa och utveckla en webbplats som fungerar på små skärmar, för att sedan kunna utöka och anpassa layouten för stora skärmar.
- **Media Queries** gör det möjligt att använda olika CSS-regler beroende på olika egenskaper hos den enhet eller miljö sidan används i. Det kan till exempel vara skärmens bredd och höjd, skärmens orientering eller om användaren använder pekskärm eller mus.
- **Relativa enheter** är måttenheter som exempelvis anpassas efter tillgängligt utrymme eller användarens textstorlek. Detta gör att sidans layout kan anpassas automatiskt.
- **CSS Flexbox och Grid** fördelar utrymme automatiskt. Flexbox används för att fördela utrymme i en dimension (horisontellt *eller* vertikalt) medan Grid kan fördela utrymme i två dimensioner (horisontellt *och* vertikalt). [5]

AJAX (Asynchronous JavaScript And XML) är en teknik för att skicka och hämta data **asynkront** från en webbserver i bakgrunden efter att en webbsida laddats in och utan att blockera annan kod från att köras samtidigt. För att göra AJAX-anrop används ofta **Fetch API**. [6]

JSON (JavaScript Object Notation) är ett textbaserat och lättläst format för utbyte och lagring av data. [6]

localStorage är ett sätt att lagra data direkt i webbläsaren som inte försvinner när sidan laddas om. [7]

Digital tillgänglighet innebär att en webbsida ska kunna användas av alla människor, inklusive personer med funktionsnedsättningar. För att uppnå detta används exempelvis semantiska element, alt-text (alternativ text) på bilder, och ARIA-attribut (Accessible Rich Internet Applications) för att förklara ett elements sammanhang för en skärmläsare. [8]

Versionshantering innebär att spåra och hålla ordning på ändringar som görs i ett projekt över tid. **Git** är ett system för att göra detta och **GitHub** är en molnbaserad webbplattform där kod kan lagras och delas. Inom Git utvecklas koden i olika **branches** (utvecklingsgrenar) för att flera personer ska kunna jobba med olika delar av koden samtidigt och för att skilja kod under utveckling från färdig och stabil kod. [9]

5 Metod

5.1 Förarbete

Arbetet kommer inledas med att fastställa en grafisk profil utifrån målgruppen. Färgpaletten tas fram med hjälp av coolors.co och typsnitten hämtas från Google Fonts samt DaFont. Bilder kommer hämtas från Unsplash och kommer sedan att komprimeras med hjälp av Photopea för att minska filstorleken. Logotyp och bilder till biljetterna kommer också att skapas i Photopea. Ikoner kommer hämtas från Font Awesome. En mockup samt wireframes för olika skärmstorlekar kommer att skapas i Figma för att fastställa en layout och utseende att jobba utifrån när webbplatsen skapas.

5.2 Innehåll och struktur

HTML kommer att användas för att skapa sidans innehåll och struktur. Semantiska HTML-element kommer att användas för att göra webbplatsen tillgänglig för besökare som exempelvis använder skärmläsare eller navigerar med tangentbordet. Bilder kommer förses med alt-texter, och ikoner som fyller en funktion utöver estetik kommer märkas med aria-labels för att användare som inte kan se webbplatsen ska förstå dess innehåll och funktion.

5.3 Layout och responsiv design

CSS kommer att användas för att utforma sidans utseende och layout. För att skapa en responsiv design kommer flera tekniker att användas:

- Media Queries kommer användas för att förändra layouten vid olika breakpoints för skärmstorlek, vilket gör att layouten kan anpassas till större skärmar [5].
- Relativa måttenheter (exempelvis rem, % och vw) kommer användas för att sidans innehåll ska skalas i förhållande till skärmstorlek och användarens textstorlek.
- CSS Flexbox kommer användas på innehåll som ska presenteras på en rad eller centreras vertikalt. CSS Grid kommer användas för layout av attraktioner och restauranger eftersom de ska presenteras i ett rutnät.

5.4 Funktionalitet

JavaScript kommer att användas för att skapa webbplatsens funktionalitet. För att köra funktioner när en viss händelse inträffar, till exempel när ett formulär skickas, kommer event handlers användas. För att spara varukorgens innehåll kommer localStorage användas eftersom det möjliggör lagring direkt i

webbläsaren som inte försvinner när sidan laddas om [7].

JSON-filer kommer att användas för att lagra data om attraktioner, restauranger och biljetter. För att hämta datan från JSON-filerna kommer Fetch API att användas eftersom det gör det möjligt att hämta data asynkront medan annan kod körs [6]. DOM-manipulation kommer användas för att uppdatera innehåll dynamiskt utan att sidan behöver laddas om.

5.5 Utvecklingsmiljö och versionshantering

Koden kommer att skrivas i Visual Studio Code och versionshanteras med Git för att spara en historik av ändringar som gjorts. Två branches kommer att användas för att separera den färdiga koden från den under utveckling. Koden kommer även skickas till ett remote repository på GitHub vilket skapar en säkerhetskopia och gör koden tillgänglig via webben [9]. Slutligen kommer webbplatsen publiceras med hjälp av GitHub Pages.

5.6 Testning

- Alla funktioner kommer att testas manuellt.
- Chrome DevTools kommer att användas för att testa webbsidan med olika skärmstorlekar.
- Olika webbläsare kommer att testas eftersom de använder olika renderingsmotorer och kan visa webbsidor på lite olika sätt [1].
- WebAIM Contrast Checker kommer användas när färgtemat tas fram för att säkerställa tillräcklig kontrast mellan färg på text och bakgrund så att all text är lättläst.
- Tangentbordsnavigering kommer att testas manuellt.
- HTML kommer att valideras med *W3C Markup Validation Service* och CSS med *W3C CSS Validation Service* för att hitta eventuella fel i koden.

6 Konstruktion

6.1 Förarbete och design

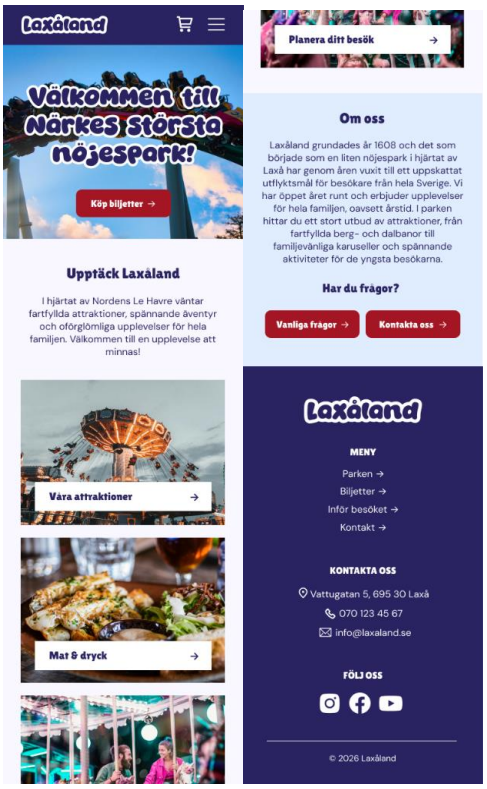
Ett färgtema med glada färger valdes utifrån målgruppen och verksamhetens karaktär. Alla färger testades med WebAIM Contrast Checker för att säkerställa tillräcklig kontrast mellan text och bakgrund. Detta är viktigt för att texten ska vara läsbar även för användare med nedsatt syn.

Tre typsnitt valdes sedan ut. För logotypen och den första rubriken på varje sida användes ett utstickande och lekfullt typsnitt, *Matcha Cih*. För knappar och underrubriker valdes *Lilita One* vilket är ett mer avskalat och läsbart typsnitt, men som fortfarande sticker ut lite. Detta byttes dock ut mot *Fredoka* senare under utvecklingen då det visade sig att *Lilita One* var svårläst på mindre text, med tjocka bokstäver och lite mellanrum. Ett tredje mycket enklare typsnitt, *DM Sans*, användes för menyer och brödtext för god läsbarhet. Alla typsnitt är sans-serif för att ge ett enhetligt utseende.

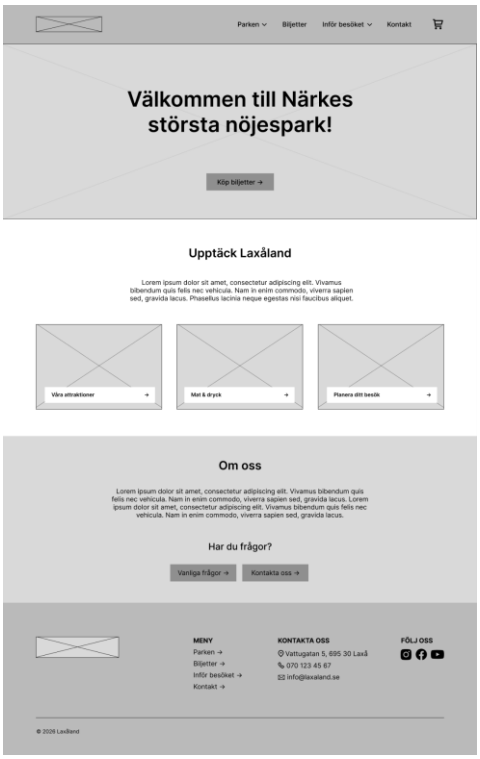
Bilder hämtades från Unsplash och med Photopea minskades bildernas storlek och sparades i formatet webp. Detta för att filstorleken ska bli så liten som möjligt, utan att förlora för mycket detaljer. Logotyp och biljettbilder designades i Photopea med samma typsnitt och färger som webbsidan för att skapa en tydlig visuell identitet till företaget.

Den hero-bild som ursprungligen valdes till startsidan byttes senare ut eftersom den visade sig se för rörig ut på större skärmar och gav dålig kontrast mellan bilden och “call to action”-knappen.

Wireframes för alla mobilsidor och en mockup för startsidan skapades med Figma enligt principen “mobile first” för att säkerställa att layouten kommer fungera för små skärmar. Sedan skapades en wireframe för startsidan på stora skärmar, för att visualisera hur layouten kan expandera när mer utrymme finns. Figur 1 visar mockupen och figur 2 visar wireframen för startsidan på stor skärm.



Figur 1 - Mockup för startsidan på liten skärm, delad på hälften



Figur 2 - Wireframe för startsidan på stor skärm

6.2 Tillgänglighet

Semantisk HTML har använts för att göra webbplatsen lättnavigerad med skärmläsare och tangentbord. Bland annat har navigationsmenyer märkts med *<nav>*, rubriker med *<h1>-<h6>,* formulär med *<form>* och länkar med *<a>*. Kartan på sidan "Inför besöket" har bäddats in från Google Maps med hjälp av *<iframe>* vilket gör den interaktiv, till skillnad från en bild [10].

Alt-texter har skrivits till alla bilder för att göra innehållet tillgängligt för användare som inte kan se webbplatsen. Till ikonerna som används för att öppna/stänga mobilmenyn, undermenyer och för att komma till företagets sociala medier har aria-labels använts. Detta har gjorts eftersom ikonerna är viktiga för att kunna använda sidans funktioner, och deras sammanhang och funktion är annars inte tydligt för en användare som inte kan se dem.

6.3 Struktur

För att ge webbplatsen en tydlig struktur har innehållet delats upp i sex stycken sidor:

- **Startsidan**: Presenterar företaget och innehåller länkar till undersidor för att leda användaren till informationen den söker.
- **Parken**: Består av sektionerna “Attraktioner” och “Mat & Dryck” som alltså visar vad som finns att göra i parken.
- **Biljetter**: Visar vilka biljettyper som finns och deras priser. Här kan användaren också lägga biljetter i varukorgen.
- **Inför besöket**: Består av sektionerna “Öppettider”, “Hitta hit” och “Frågor & svar” vilka ger användaren den information som behövs inför ett besök.
- **Kontakt**: Innehåller kontaktuppgifter och ett kontaktformulär.
- **Varukorg**: Visar biljetter som användaren lagt i varukorgen, totalsumman för biljetterna och ger användaren möjlighet att ta bort biljetter.

6.4 Navigation

För enkel navigation på stora skärmar har en horisontell meny i sidhuvudet skapats, med utfällbara undermenyer för sidor med flera typer av innehåll. På små skärmar ersätts denna av en vertikal meny som går att öppna och stänga. Detta har gjorts för att spara plats när utrymmet blir mer begränsat, men också för att skapa mer utrymme runt länkarna eftersom små skärmar ofta har pekskärm. En enklare meny utan underrubriker har också skapats i sidfoten.

För sidorna “Parken” och “Inför besöket” som innehåller flera sektioner med olika innehåll har även en enkel meny för sidnavigation skapats. Sidnavigationen består av knappar positionerade direkt under sidans hero. Knapparna består av länkar som leder till ett element med ett visst id. Varje sektion på sidan har givits ett unikt id som länkarna i sidnavigationen, såväl som länkarna i undermenyerna från förra stycket, leder till.

6.5 Utseende och layout

För att skapa webbplatsens layout har CSS Flexbox använts på många ställen, till exempel i huvudmenyn, puffarna på startsidan, sidfoten och biljetterna. Detta gjordes för att innehållet skulle kunna presenteras horisontellt på en rad. På små skärmar där innehållet i dessa sektioner istället skulle presenteras i en kolumn applicerades CSS-regeln *flex-direction: column*.

Innehållet på sektionerna “Attraktioner” och “Mat & Dryck” presenteras i ett rutnät på stora skärmar och därför användes CSS Grid.

För att applicera olika CSS-regler beroende på skärmstorlek har Media Queries använts. En primär *breakpoint* vid 900px har använts för att anpassa layouten till stora skärmar. På skärmar bredare än 900px visas bland annat huvudmenyn och sidfoten på en rad istället för en kolumn. Breakpoints vid 1200px och 1400px sattes också för att utöka avståndet mellan sidans kanter och närmsta element vilket gör innehållet mer kompakt på mycket stora skärmar. På sidan “Parken” sattes även en breakpoint vid 600px för att justera antal kolumner i dess CSS Grids.

Figur 3 visar hur CSS text-shadow användes för att skapa en kant runt huvudrubrikerna på varje sida. Detta var viktigt för att skapa kontrast och göra texten läsbar ovanpå bakgrundsbilden. Det visade sig dock vara mycket omständligt att skapa en tjock kant runt texten, och den färdiga lösningen är inte min egen utan hämtades från forumet Stack Overflow [11].



Figur 3 - Huvudrubrik med vit kant

6.6 Funktionalitet

6.6.1 Validering av kontaktformulär

För att säkerställa att kontaktformuläret är korrekt ifyllt valideras det med JavaScript. Detta gör att användaren kan få alla felmeddelanden samtidigt när den skickar formuläret. Det hindrar också användaren från att manipulera reglerna för inmatning genom att exempelvis radera ett “required”-attribut, vilket annars är möjligt om endast den inbyggda HTML-valideringen används [7]. Valideringen görs när formuläret skickas genom en *event listener* som anropar en funktion som kontrollerar att alla fält är ifyllda. Om något fält är tomt skrivs ett felmeddelande ut under fältet och en CSS-klass appliceras på inmatningsfältet vilket ger det en röd kant.

6.6.2 Visa och dölja innehåll

I mobilmenyn, undermenyerna och svaren till "Frågor & svar" används JavaScript för att visa och dölja innehåll. Detta har gjorts genom att en *event listener* registrerar att knappen för att exempelvis öppna mobilmenyn har klickats på. Då anropas en funktion som lägger till CSS-klassen "open" på det dolda elementet, vilket i sin tur triggar CSS-regler som gör att elementet visas. När knappen sedan klickas på igen tas istället klassen "open" bort, och elementet döljs.

6.6.3 Spara biljetter till varukorg

Alla biljetter har försetts med en knapp för att lägga biljetten i varukorgen. Denna knapp har också kopplats till en *event listener* i JavaScript. När knappen klickas på anropas en funktion som uppdaterar varukorgens innehåll, som är sparat i localStorage, med den nya biljetten.

Utöver detta har en "animation" lagts till på biljetternas "köp nu"-knappar. När knappen klickas på blir den grön och texten byts ut mot en bock, men strax därefter återställs knappen. Detta har åstadkommits genom att först ändra text och applicera en CSS-klass för att ändra färg. Sedan användes en setTimeout() metod som anropar en funktion för att ta bort klassen och ändra tillbaks texten efter ett visst tidsintervall, i detta fall 700 ms.

6.6.4 Dynamiskt innehåll

Attraktioner, restauranger och biljetter har inte lagts in i HTML-dokumenten från början utan lagras istället i JSON-filer. För att hämta JSON-datan asynkront när sidan redan är laddad används *Fetch API*. Därefter renderas innehållet med hjälp av DOM-manipulation, där elementen skapas och skrivs ut efter en gemensam mall. Detta gjorde det enkelt att förändra utskriften genom att bara ändra på ett ställe i JavaScript-koden istället för att behöva ändra på alla HTML-element.

Varukorgens innehåll visas också med hjälp av DOM-manipulation, men hämtas från localStorage. Varje biljett i varukorgen har en knapp som används för att ta bort biljetten. När den klickas på så tas den aktuella biljetten bort från localStorage. Sedan anropas funktionen för att rendera varukorgens innehåll igen, vilket gör att sidan inte behöver uppdateras när en biljett tagits bort.

6.7 Versionshantering och publicering

Git har använts för att versionshantera koden till webbplatsen. En "dev"-branch har använts för att utveckla koden, vilken sedan har sammanslagits med

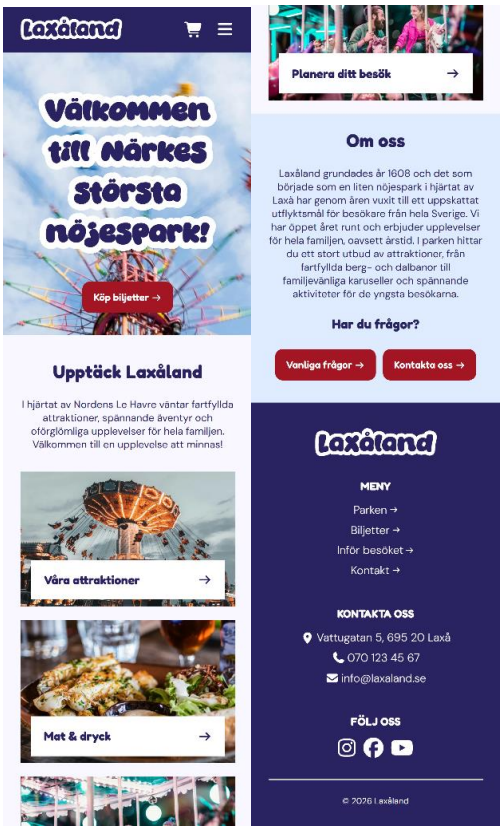
huvudgrenen *main* när en funktion varit färdig och fungerande. Detta har gjorts för att separera halvfärdig och instabil kod från den färdiga som används på den publicerade webbplatsen. Koden har också *pushats* kontinuerligt till ett *remote repository* på GitHub för extern lagring och säkerhetskopiering. Webbplatsen publicerades med GitHub Pages och uppdateras automatiskt när ny kod *pushas* till *main*.

6.8 Testning

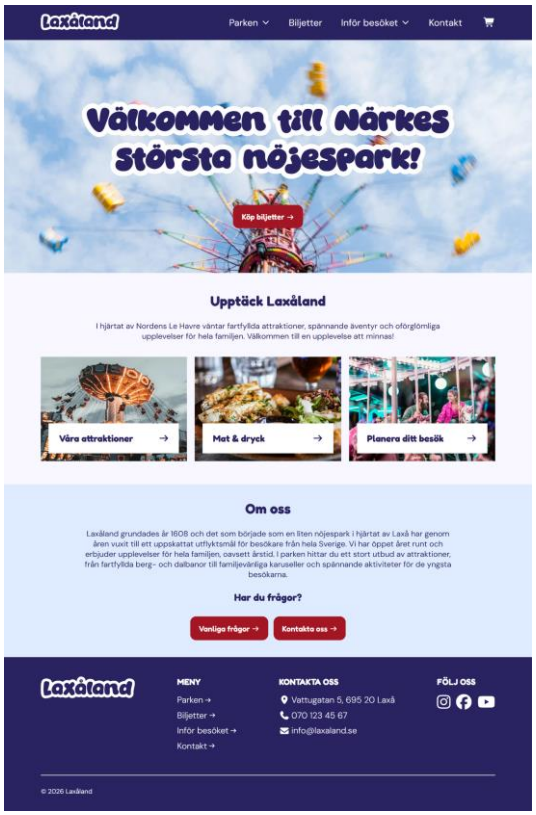
- Alla funktioner har testats manuellt.
- Chrome DevTools har använts för att testa webbplatsen med olika skärmstorlekar.
- Webbplatsen har testats i webbläsarna Google Chrome, Microsoft Edge och Mozilla Firefox.
- WebAIM Contrast Checker har använts när färgtemat togs fram.
- Tangentbordsnavigering har testats manuellt.
- HTML har validerats med W3C Markup Validation Service och CSS med W3C CSS Validation Service.

7 Resultat

Den färdiga webbplatsen innehåller information om parken, dess öppettider, vägbeskrivning, vanliga frågor och kontaktuppgifter. Figur 4 och 5 visar hur webbplatsens layout anpassas efter skärmstorleken.



Figur 4 - Startsidan på liten skärm, delad på hälften



Figur 5 - Startsidan på stor skärm

Den funktionalitet som finns är:

- En mobilmeny som går att öppna och stänga.
- Undermenyer till huvudmenyn som går att öppna och stänga.
- Svar på vanliga frågor som går att visa och dölja.
- Ett kontaktformulär som valideras med JavaScript.
- En sida för biljettköp finns där användaren kan lägga biljetter i varukorgen.
- En varukorg finns där biljetter visas och kan tas bort.

Resultat av tester som gjorts:

Test	**Förväntat resultat**	**Resultat**
Test på olika skärmstorlekar	Innehållet anpassas till skärmbredden	Godkänt
Öppna och stäng mobilmeny	Menyn öppnas och stängs vid klick	Godkänt
Öppna och stäng undermenyer	Undermenyer öppnas och stängs vid klick	Godkänt
Skicka tomt/delvis ifyllt kontaktformulär	Felmeddelanden visas	Godkänt
Visa och dölj svar på frågor i FAQ	Svaren visas och döljs vid klick	Godkänt
Visa dynamiskt innehåll	Innehållet visas	Godkänt
Använda länkar	Länken leder till förväntad sida	2 fel upptäcktes
Spara biljetter i varukorgen	Knappen animeras och biljetter finns i varukorgen	Godkänt
Ta bort biljetter ur varukorgen	Biljetten försvinner ur varukorgen och totalsumman minskar med biljettens pris	Godkänt
HTML- och CSS-validering	Inga valideringsfel	Godkänt
Kontrast mellan text och bakgrund	Ratio på minst 4.5:1 för att uppfylla WCAG 2 nivå AA	Godkänt
Olika webbläsare	Sidan ser i stort sett likadan ut och fungerar korrekt	Godkänt
Tangentbordsnavigering	Navigationen är logisk och alla funktioner går att använda med endast tangentbord	Godkänt

Alla tester ger förväntat resultat förutom länkarna som går till sektionerna “Attraktioner” och “Mat & dryck”. Istället för att hamna på rätt sektion hamnar man ibland på toppen av sidan “Parken”, alltså rätt sida men inte rätt sektion.

8 Slutsatser

8.1 Måluppfyllelse

Målet var att skapa en webbplats där Laxåland kan nå ut med information till kunder som vill besöka parken. Sidan skulle fungera på olika skärmstorlekar och vara tillgänglig för besökare med funktionsnedsättning. Dessutom skulle den ha följande funktionalitet:

- En mobilmeny som går att öppna och stänga.
- Undermenyer till huvudmenyn som går att öppna och stänga.
- Svar på vanliga frågor som går att visa och dölja.
- Ett kontaktformulär som valideras med JavaScript.
- En sida för biljettköp finns där användaren kan lägga biljetter i varukorgen.
- En varukorg finns där biljetter visas och kan tas bort.

Den färdiga webbplatsen innehåller all information och funktionalitet som planerades och uppfyller därför målet.

8.2 Förklaring till problem

Problemen med länkarna till sektionerna “Attraktioner” och “Mat & Dryck” som inte alltid leder till dessa, utan till toppen av sidan, beror på att innehållet renderas dynamiskt. Innan innehållet har renderats har sektionerna ingen höjd. Länkarna leder alltså egentligen till rätt sektion, men eftersom inget innehåll finns så är båda sektionerna längst upp på sidan. Om innehållet redan är inladdat fungerar länkarna som förväntat.

8.3 Diskussion kring dynamiskt innehåll

Att läsa in attraktioner, restauranger och biljetter från en JSON-fil och skapa innehållet med DOM-manipulation fyller egentligen ingen funktion för användaren just nu. Min tanke var att jag i mån av tid också skulle ge användaren möjlighet att filtrera exempelvis attraktioner baserat på längd. Jag tänkte att detta skulle bli enklare att göra om jag hade allt sparat som färdiga objekt. Detta hann jag dock inte implementera. Fördelen med att rendera exempelvis attraktioner och restauranger med DOM-manipulation är dock att allt kan skrivas ut på samma sätt och det räcker att ändra JavaScript-koden på ett ställe om man vill förändra utskriften, istället för att “hårdkoda” det i HTML och behöva ändra alla element. Däremot fyller dynamisk utskrift med DOM-manipulation en funktion i varukorgen där biljetter ska kunna tas bort och varukorgen uppdateras utan att användaren behöver ladda om sidan.

8.4 Förbättringsmöjligheter

- Just nu är bara själva orden i huvudmenyn klickbara, men det hade varit lättare att komma åt länken om hela området runt också varit länkat. Detta är främst ett problem i mobilmenyn eftersom man måste sträcka tummen långt för att komma åt länken.
- Det hade varit bra om undermenyer öppnade sig när muspekaren "hovrar" över huvudrubriken i menyn. Som det är nu så måste man klicka på pilen bredvid för att fälla ut undermenyn, vilket funkar bra på mobilmenyn men gör det svårare på datorn eftersom man måste klicka exakt på pilen.
- JavaScript-koden innehåller en del funktioner som gör ungefär samma sak, till exempel finns flera funktioner för att hämta data från JSON-filer. För att få en mer kompakt och lättläst kod hade det varit bra att "baka ihop" vissa funktioner.
- Alla undersidor har samma hero-bild. Det hade varit snyggare om de hade olika.

8.5 Vidareutveckling

- En varukorg där flera av samma biljett visas som ett (1) element med en text som visar hur många det är och möjlighet att öka eller minska antalet.
- Klickbara attraktioner och restauranger, så att en ny sida med mer information visas.
- Filtrering av längd på karuseller.
- Sida för betalning och möjlighet att faktiskt lägga en beställning.

Källförteckning

[1] Mittuniversitetet. Introduktion till HTML [videofil]. 2026-08-19 [citerad 2026-10-29]. Hämtad från: https://youtu.be/zx1wKb1cehE?si=D-8JxdPbkHeIYJnm

[2] MDN Web Docs. What is CSS? [internet]. Mozilla Corporation; c2026 [uppdaterad 2026-07-09; citerad 2026-10-29]. Hämtad från: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/What_is_CSS

[3] Mittuniversitetet. JavaScript [videofil]. 2026-08-26 [citerad 2026-10-29]. Hämtad från: https://youtu.be/UVc3rxWSjuw?si=w8AeP-eZe5NJF509

[4] Mittuniversitetet. DOM, events & formulärhantering [videofil]. 2026-06-11 [citerad 2026-11-01]. Hämtad från: https://youtu.be/-xX8yW6_Svc?si=TQ_SmioNstDjyzJj

[5] Mittuniversitetet. Responsivitet, layout och positionering [videofil]. 2026-09-09 [citerad 2026-11-01]. Hämtad från: https://youtu.be/7kK9oqRM-5A?si=-EHXHR7TPwKu6yZH

[6] Mittuniversitetet. Asynkron programmering & AJAX [videofil]. 2026-06-17 [citerad 2026-11-01]. Hämtad från: https://youtu.be/wR4hapvCMCo?si=5UeSCVlparTIXn9z

[7] Mittuniversitetet. JavaScript - Validering, felhantering och storage [videofil]. 2026-10-01 [citerad 2026-11-01]. Hämtad från: https://youtu.be/8OKj7XNzgQ4?si=zgveuLGBDOhCz0RO

[8] W3C Web Accessibility Initiative. How to Meet WCAG (Quick Reference) [internet]. World Wide Web Consortium; c2025 [uppdaterad 2026-09-25; citerad 2026-11-01]. Hämtad från: https://www.w3.org/WAI/WCAG22/quickref/

[9] Mittuniversitetet. Git - Versionhantering [videofil]. 2026-08-23 [citerad 2026-11-01]. Hämtad från: https://youtu.be/rPHn4vwFJrk?si=ehWEkcRIbUm7FF0y

[10] MDN Web Docs. <iframe> HTML inline frame element [internet]. Mozilla Corporation; c2026 [uppdaterad 2026-10-01; citerad 2026-11-01]. Hämtad från: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/iframe

[11] user128511. Outline effect to text [Internet]. Stack Overflow; 11-03-

2019 [uppdaterad 2022-10-20; citerad 2026-11-01]. Hämtad från: https://stackoverflow.com/a/55097644